

TRP 1 – AI Content Generation Challenge: Submission Report

Candidate: Hiwot Beyene

Date: February 2, 2026

1. Environment Setup Documentation

APIs I configured

I set up the following in `trp1-ai-artist/.env`:

- **GEMINI_API_KEY** – From Google AI Studio (aistudio.google.com). I use it for Lyria (music) and Veo (video).
- **AIMLAPI_KEY** – From AIMLAPI.com, for MiniMax music with vocals.
- **KLINGAI_API_KEY** and **KLINGAI_SECRET_KEY** – From Kling AI, for Kling video generation (optional for the challenge).

Issues during setup and how I resolved them

1. `uv` not found

After cloning and running `uv sync`, the shell said `uv` was not installed. The system suggested `sudo snap install astral-uv`, which failed because the snap uses classic confinement. I ran `sudo snap install astral-uv --classic` and then opened a new terminal so `uv` was on PATH. After that, `uv sync` and `uv run ai-content --help` worked.

2. `.env` creation

I ran `cp .env.example .env` in `trp1-ai-artist` and edited `.env` in Cursor to paste my API keys (no quotes, no extra spaces). I made sure to run all commands from the project root so the app could find `.env`.

2. Codebase Understanding

Architecture (from `docs/architecture/ARCHITECTURE.md` and my exploration)

The app is built around a **ProviderRegistry** and protocol-based providers:

- **CLI** (`src/ai_content/cli/main.py`) – Typer app. Commands like `music`, `video`, `list-providers`, `list-presets` call the registry and run providers.
- **ProviderRegistry** – Holds music, video, and image providers. Providers register with decorators (e.g. `@ProviderRegistry.register_video("veo")`). The registry returns the right provider by name.
- **Providers** – Each implements a protocol (`MusicProvider`, `VideoProvider`, or `ImageProvider`) with `name` and `async def generate(...) -> GenerationResult`. Music: `lyria` (Google), `minimax` (AIMLAPI). Video: `veo` (Google), `kling` (KlingAI). Image: `imagen` (Google).

- **Presets** – `src/ai_content/presets/music.py` and `video.py` define style presets (e.g. jazz, nature) with prompt, BPM/mood or aspect ratio. The CLI uses `--style` to apply a preset and overwrite prompt/BPM or aspect ratio.
- **Pipelines** – `src/ai_content/pipelines/` (e.g. `music.py`, `video.py`, `full.py`) orchestrate multi-step flows (e.g. music then image then video then merge). The CLI can call providers directly or pipelines can call them for full workflows.
- **Config** – Pydantic settings in `config/settings.py` load from `.env` and optional YAML. Nested models (e.g. `KlingSettings`) can use a different env prefix, which caused issues for me with Kling (see Challenges).

Generated files go to `exports/` (or a path you pass with `--output`).

Insights about the provider system

- Adding a provider = new class + `@ProviderRegistry.register_*(name)` + import so it loads. No need to edit a central list.
- All providers return `GenerationResult` (success, file_path, error, etc.), so the CLI and pipelines handle results in a uniform way.
- Music requires `--prompt` (or `-p`); `--style` is optional and, when set, replaces the prompt with the preset's prompt. Same idea for video.

How pipeline orchestration works

Pipelines (e.g. `MusicPipeline`, `FullMusicVideoPipeline`) get a provider from the registry, optionally resolve a preset for prompt/BPM or aspect ratio, call `provider.generate(...)`, and collect `GenerationResults`. The full pipeline in `full.py` can do music → image → video → FFmpeg merge; the CLI mainly drives single providers (music or video) with optional presets.

3. Generation Log

Commands I ran

Verification:

```
uv run ai-content --help
uv run ai-content list-providers
uv run ai-content list-presets
```

Music (instrumental – Lyria):

```
uv run ai-content music -p "jazz" -s jazz --provider lyria -d 30
```

- **Prompt (effective):** From the jazz preset (smooth jazz fusion, walking bass, brushed drums, etc.).
- **Result:** Success. Output in `exports/` (e.g. `lyria_20260202_125141.wav`). I used this as my main generated audio artifact.

Music with vocals (MiniMax + lyrics):

```
uv run ai-content music -p "Soul, warm vocals" --provider minimax --lyrics lyrics.txt -d 30
```

- **Result:** Failed with 403 Forbidden – AIMLAPI requires “Complete verification to using the API” (err_unverified_card). I couldn’t complete payment/card verification, so I did not get any vocal music from MiniMax (see Challenges).

Video (Veo):

```
uv run ai-content video -p "nature" -s nature --provider veo -d 5
```

- **Result:** 429 RESOURCE_EXHAUSTED after retries. No video file produced.

Video (Kling):

```
uv run ai-content video -p "nature" -s nature --provider kling -d 5
```

- **Result:** 429 Too Many Requests after retries. No video file produced.

Results summary

Content	Provider	Outcome	Artifact / note
Instrumental audio	Lyria	Success	exports/lyria_*.wav (e.g. ~30 s)
Audio with vocals	MiniMax	Blocked	403 – verification (card) required; I couldn’t do it
Video	Veo	Failed	429 after 4 retries – quota exceeded
Video	Kling	Failed	429 after 4 retries – rate limit / quota

I did not achieve a generated video or a combined music video because both Veo and Kling returned 429 for my account.

4. Challenges & Solutions

Audio / music

1. **Missing option --prompt / -p**
I ran `uv run ai-content music --style jazz --provider lyria` and got “Missing option ‘--prompt’ / ‘-p’”. I checked `cli/main.py`: the `music` command requires `--prompt` (or `-p`). When using a preset, you still have to pass a prompt; the preset then overwrites it. I used `-p "jazz" -s`

jazz so the preset's prompt and BPM were applied. Same idea for video: `-p "nature" -s nature`.

2. Lyrics file "Loaded lyrics: 0 characters"

First time I ran the MiniMax command with `--lyrics lyrics.txt`, the log said "Loaded lyrics: 0 characters" and the API complained that lyrics were required. My `lyrics.txt` had content in the editor but hadn't been saved, or the process was reading from the wrong path. I saved `lyrics.txt` in `trp1-ai-artist` with real lyrics (verse/chorus structure), ran again from the project root, and the CLI then reported hundreds of characters loaded. So the fix was: save the file and run from the correct directory.

3. MiniMax 403 Forbidden – verification required

After fixing the lyrics file, the request to AIMLAPI returned 403 with "Complete verification to using the API" and `err_unverified_card`. I went to the URL in the error (aimlapi.com/app/verification). The flow requires payment/card verification. I wasn't able to complete that step, so I couldn't use MiniMax for vocals. I documented this as a blocker rather than a code bug: the integration is correct, but account verification is required and I didn't have a way to complete it.

Video

4. Veo: `GenerateVideoConfig` not found

Veo failed with "module 'google.genai.types' has no attribute 'GenerateVideoConfig'". I looked at the Google video-generation docs and the installed SDK: the correct type is `GenerateVideosConfig` (with an "s"), and the method is `generate_videos` (plural). I updated `veo.py` to use `GenerateVideosConfig` and `generate_videos` instead of `GenerateVideoConfig` and `generate_video`.

5. Veo: `person_generation` "allow_adult" not supported

After fixing the config name, Veo returned 400: "allow_adult for personGeneration is currently not supported." The docs say for text-to-video the only supported value is "allow_all". I changed the default in `veo.py` from `allow_adult` to `allow_all` so text-to-video requests use the supported value.

6. Veo and Kling: 429 (quota / rate limit)

Both Veo and Kling started returning 429 (Too Many Requests / RESOURCE_EXHAUSTED). I looked up recommended handling: exponential backoff and retry. I added retry logic in both providers: up to 4 attempts, with delays 15s, 30s, 60s, 120s (capped), and only retry when the error is 429 (or RESOURCE_EXHAUSTED for Veo). The code now retries as intended (I see "Veo rate limited (429). Retrying in 15s (attempt 1/4)..." etc.), but after all 4 attempts both APIs still return 429. So the approach is correct for transient rate limits, but in my case the quota/limit is fully exhausted (daily or plan limit), not just a short burst. Retrying later or upgrading the plan is the only fix on the API side; there's no further code change that would make the request succeed with my current quota.

7. Kling: "Authentication failed. Check API key."

Even with what I thought were correct keys in `.env`, Kling raised "Authentication failed. Check API key." I checked how settings are loaded: for nested Pydantic settings (e.g. `kling`), the env vars can be resolved with a different prefix (e.g. `KLING_*` instead of `KLINGAI_*`), so `KLINGAI_API_KEY` and `KLINGAI_SECRET_KEY` weren't being picked up. I added a fallback in the Kling provider: if the loaded settings have empty `api_key` or `secret_key`, read from `os.environ` for `KLINGAI_API_KEY`

and `KLINGAI_SECRET_KEY`, and if still empty call `load_dotenv()` and try again. After that, Kling authenticated and I got 429 instead of 401/403, confirming the keys and fallback work.

5. Insights & Learnings

- **CLI design:** Requiring `--prompt` even when using `--style` felt redundant at first, but the code applies the preset only when both are present; the prompt is the required “seed” and the preset overwrites it. I’d add a short note in `--help` or the docs: “When using `--style`, you can pass a placeholder prompt (e.g. `-p jazz`) and the preset will override it.”
 - **Nested settings and env names:** Pydantic-settings’ behavior for nested models (env prefix derived from the parent key) meant my `.env` names didn’t match what the Kling settings loader expected. I’d either document the exact env var names per nested model or, as we did, add a fallback for the “expected” names (e.g. `KLINGAI_*`) so both conventions work.
 - **429 handling:** Retry with exponential backoff is the right approach and matches Google’s and others’ guidance. It helps when the limit is per-minute or transient; it doesn’t help when the account has no remaining quota. Clearer error messages (e.g. “quota exhausted for today” vs “rate limited, retry in a minute”) would make it easier to decide whether to retry or wait longer.
 - **MiniMax / AIMLAPI verification:** Needing card verification to use the API is an account/policy constraint, not something fixable in this repo. I’d note it in the challenge instructions so others know vocal music may require completing verification on AIMLAPI’s side.
 - **Comparison to other tools:** This codebase is well-structured (registry, protocols, async, clear separation of CLI vs providers vs pipelines). It’s easier to add a new provider here than in a single monolithic script. The main friction I hit was env/keys (nested settings, verification) and API quotas rather than the code itself.
-

6. Links

- **YouTube:** I did not upload a video because I could not generate a successful video file; both Veo and Kling returned 429 after retries.
 - **GitHub repo (exploration / submission):** <https://github.com/Hiwot-Beyene/TRP-1---AI-Content-Generation-Challenge---Second-Challenge>
-

Summary

I completed environment setup (Gemini, AIMLAPI, Kling keys; `uv` install), explored the codebase (registry, providers, presets, pipelines), and successfully generated **instrumental audio** with Lyria. I was **blocked on audio with vocals** by AIMLAPI’s payment/card verification requirement, which I couldn’t complete. I fixed several **video** issues in code (Veo config/method names and `person_generation`; Kling env fallback; 429 retry with backoff for both), but **video generation still failed** with 429 after all retries because my quota/rate limit is exhausted on both Veo and Kling. I documented the approach for each fix and the limits (verification, quota) that are outside the codebase.